



Storitveno usmerjene arhitekture

izr. prof. dr. Boštjan Šumak

1

Nazadnje

- Arhitekture mikrostoritev
- Karakteristike mikrostoritev in mikrostoritvene arhitekture
- Prednosti mikrostoritev
- Izzivi pri razvoju na osnovi mikrostoritev
- SOA vs mikrostoritvena arhitektura

2

Mikrostoritve : izzivi in rešitve

3

Ključni izzivi pri razvoju mikrostoritev

- Kako sistem smiselno **razdeliti na mikrostoritve**?
- Kako storitve **uvajati in izvajati** (konfiguracija, CI/CD)?
- Kako zagotoviti učinkovito komunikacijo:
 - **odjemalec** $\leftrightarrow$ **storitve** (API, usmerjanje, prehodi),
 - **storitev** $\leftrightarrow$ **storitev** (sinhrono/asinhrono, pogodbe)?
- Kako obvladovati **izpade in delne okvare**, da ne ohromijo celotnega sistema (odpornost na napake)?
- Kako upravljati **porazdeljene podatke** in zagotoviti **konsistentnost**?
- Kako izvajati **integracijsko testiranje** in preverjati pogodbe med storitvami?
- Kako koordinirati **spremembe čez več povezanih storitev**?
- ...

4

Kako reševati izzive pri razvoju mikrostoritev?

Vzorci, dobre prakse

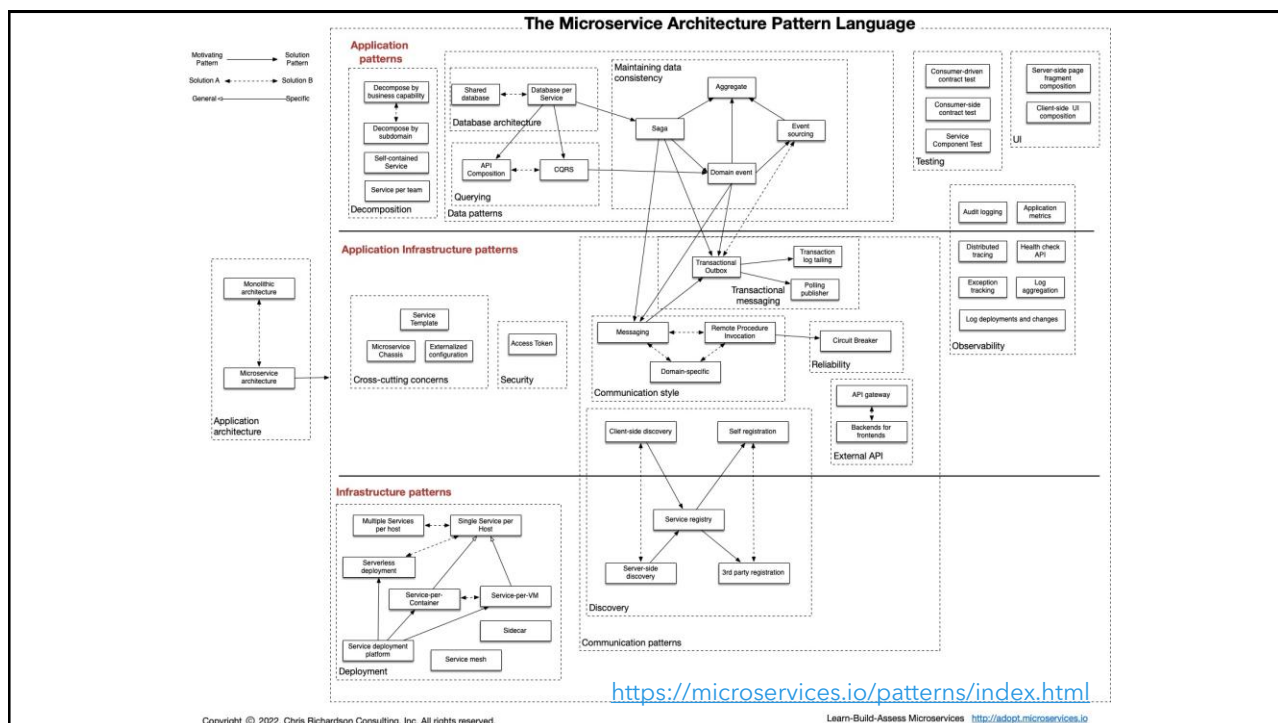


Programska ogrodja, orodja in platforme

- API prehod (API Gateway)
- Register storitev (Service Registry / Service Discovery)
- Circuit Breaker (za odpornost na napake)
- Event Sourcing
- CQRS
- ...

- knjižnice in moduli (npr. za komunikacijo, odpornost, opazljivost)
- zmožnosti oblačnih platform (npr. upravljanje konfiguracije, skaliranje, nadzor)
- zabojniki in izvajalna okolja (npr. Docker) ter orkestracija (npr. Kubernetes)
- ...

5



6

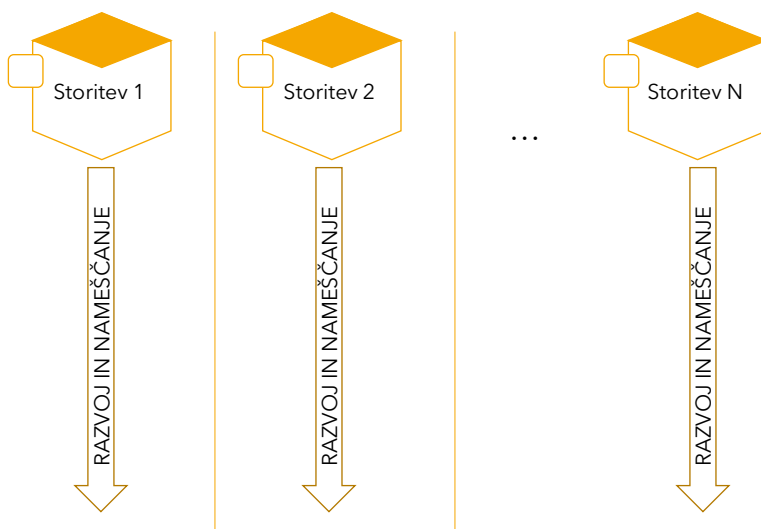
Kako razdeliti sistem na mikrostoritve?

• Osnovna načela:

- Arhitektura mora imeti **stabilne meje med storitvi** (jasno določene domene in odgovornosti).
- Vsaka storitev naj zagotavlja **majhen nabor tesno povezanih funkcionalnosti** (*ena poslovna zmožnost ali ozek del domene*).
- Storitve naj bodo **šibko sklopljene**: storitev navzven izpostavi API, notranjo implementacijo pa lahko spreminjamo brez vpliva na odjemalce (*ob ohranjanju pogodbe/združljivosti*).
- Storitev naj bo dovolj omejena, da jo lahko učinkovito razvija in vzdržuje **majhna ekipa** (*približno 6-10 članov*).
- Ekipe morajo biti **avtonomne**: sposobne razviti, testirati in uvesti svoje storitve z minimalnim usklajevanjem z drugimi ekipami..

7

Osnovni cilj dekompozicije

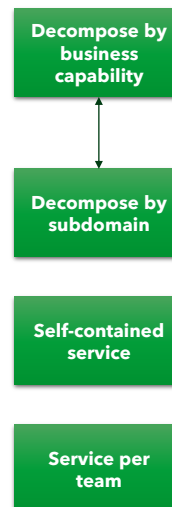


omogočiti neodvisen&vzporeden razvoj in nameščanje storitev

8

Vzorci dekompozicije sistema na mikrostoritve

- Dekompozicija po poslovnih zmožnostih
- Dekompozicija po domenah/poddomenah
- Dekompozicija na samozadostne storitve
- Dekompozicija po odgovornostih ekip



Povzeto/prirejeno po: <https://microservices.io/patterns/index.html>

9

Dekompozicija po poslovnih zmožnostih

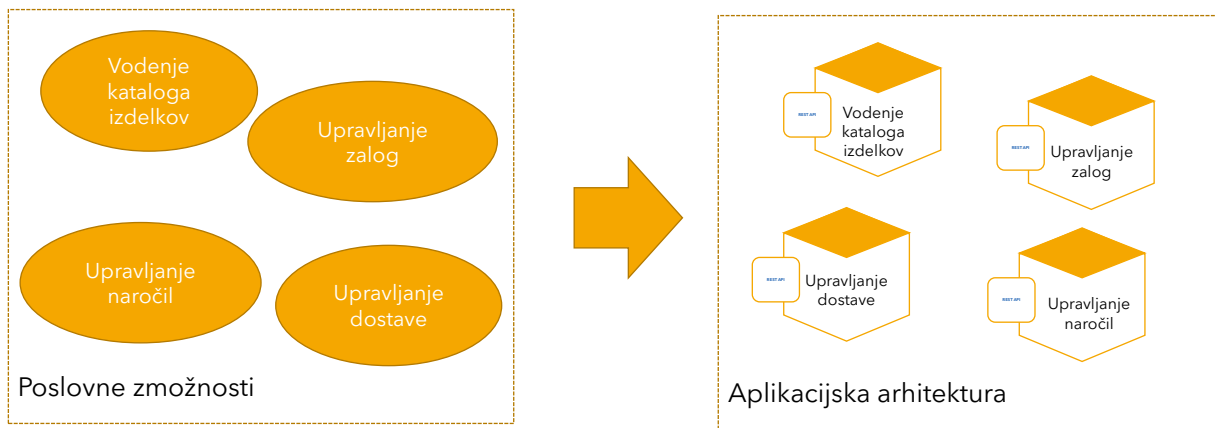
- Storitve določimo glede na **poslovne zmožnosti** (*business capabilities*).
- Poslovne zmožnosti opredelimo v okviru **modeliranja poslovne arhitekture**.

Poslovna zmožnost je dejavnost, ki jo organizacija izvaja, da ustvari vrednost.

- Pogosto je povezana z določenim **poslovnim objektom (entiteto)**, npr.:
 - upravljanje naročil → naročila,
 - upravljanje strank → stranke,
 - ...
- Primer spletne trgovine (možne zmožnosti/storitve):
 - upravljanje kataloga izdelkov,
 - upravljanje zalog,
 - upravljanje naročil,
 - upravljanje dostave,
 - ...

10

Dekompozicija po poslovnih zmožnostih



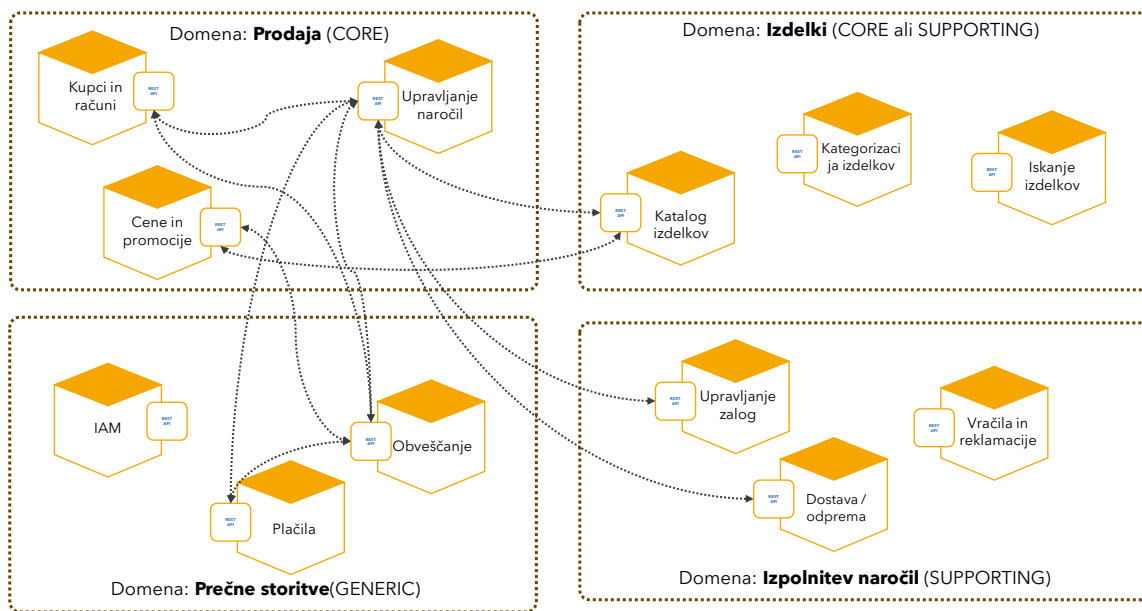
11

Dekompozicija po domenah/poddomenah

- Poslovanje organizacije lahko običajno razdelimo na domene, posamezna domena pa je sestavljena iz več poddomen.
 - Vsaka poddomena predstavlja določen del poslovanja in je dober kandidat za jasno mejo pri zasnovi storitev.
- Primer delitve poddomen:
 - **Jedrna (core) poddomena:** ključna za konkurenčno prednost podjetja; najdragocenejši del sistema.
 - **Podporna (supporting) poddomena:** pomembna za delovanje, vendar ni ključna; lahko se izvaja interno ali se odda zunanjim izvajalcem.
 - **Splošna (generic) poddomena:** ni specifična za podjetje; pogosto jo je smiselno rešiti z obstoječimi standardnimi rešitvami.

12

Dekompozicija po domenah/poddomenah (primer)



13

Dekompozicija na samozadostne storitve

- Posamezna storitev mora zagotavljati **visoko razpoložljivost in nizek odzivni čas**.
- V mikrostoritvah se obdelava ene zahteve pogosto razdeli na več storitev, kar pri **sinhronih klicih** poveča tveganje za zakasnitve in izpade.
- **Razpoložljivost operacije** je v takem primeru odvisna od **razpoložljivosti vseh storitev**, ki sodelujejo v obdelavi.
- Zato storitve načrtujemo čim bolj **samozadostno**
 - naj bodo sposobne odgovoriti na **sinhrone zahteve z minimalno odvisnostjo** od drugih storitev (kjer je mogoče).

14

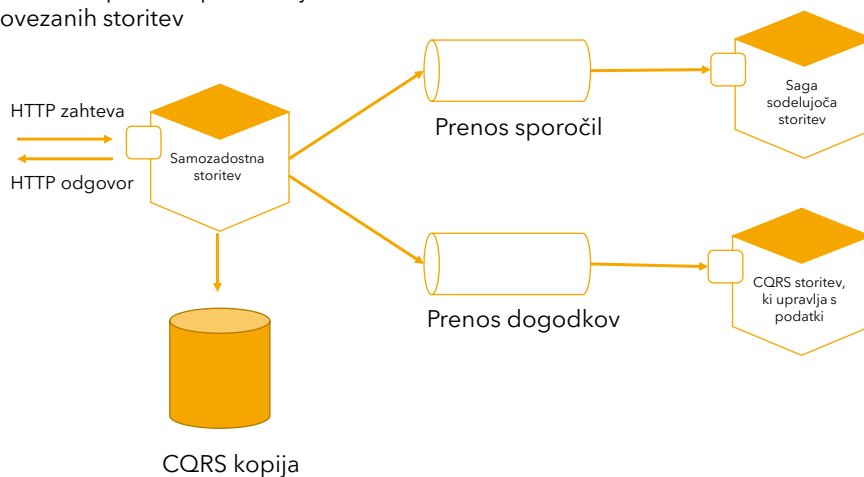
Dekompozicija na samozadostne storitve

- Ena možnost je zmanjšati odvisnost od sinhronih klicev in sodelovanje med storitvami zasnovati **asinhrono**
 - npr. z vzorcema **Saga** in **CQRS**
- **Saga** omogoča upravljanje poslovnih procesov prek zaporedja lokalnih transakcij in **časoma konsistentno** usklajevanje podatkov
 - po potrebi tudi s kompenzacijami
- **CQRS** omogoča ločitev zapisovanja in branja ter uporabo **projekcij/replik** podatkov, ki izvirajo iz drugih storitev
 - npr. za hitrejšo in bolj samozadostno branje

15

Dekompozicija na samozadostne storitve

Storitev zgradimo tako, da lahko odgovori na sinhroni klic brez potrebe po čakanju na odgovor povezanih storitev



Povzeto/prirejeno po: <https://microservices.io/patterns/decomposition/self-contained-service.html>

16

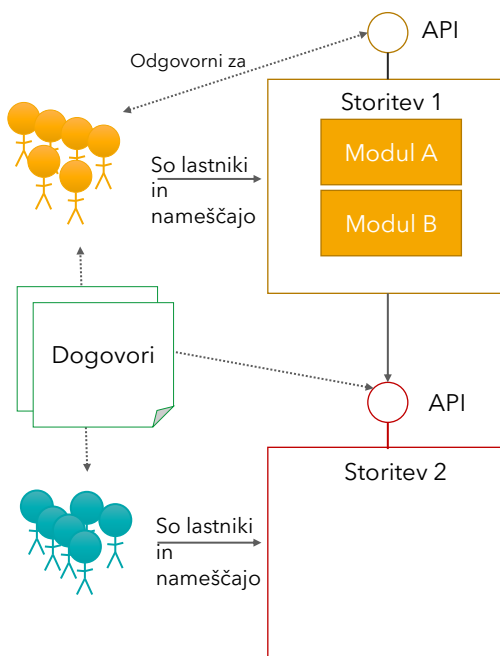
Dekompozicija po odgovornostih ekip

- Storitve razmejujemo tako, da jih lahko razvija in vzdržuje **majhna ekipa (5-9 članov)**
 - Ekipa naj bo **avtonomna** in z drugimi ekipami **šibko povezana** (minimalna odvisnost, jasno definirani vmesniki)
 - Obseg in kompleksnost kode v domeni ekipe naj ne presegata **kognitivne zmožnosti** ekipe (da je sistem še obvladljiv).
- **Drobnozrnata dekompozicija** lahko izboljša:
 - **vzdrževanje,**
 - **testiranje,**
 - **neodvisno uvajanje (deployment),**
 - in **pospeši razvojne cikle.**

17

Dekompozicija po odgovornostih ekip

Velikost/kompleksnost, ki jo obsega storitev ne sme preseči kognitivnih sposobnosti ekipe



Povzeto/prirejeno po: <https://microservices.io/patterns/decomposition/service-per-team.html>

18

Dekompozicija: koliko storitev?

- *Premalo storitev* pogosto pomeni, da se sistem **ščasoma približa monolitu**
 - začne kazati podobne težave (kompleksnost, težje uvajanje, manjša avtonomija, ...)
- *Preveč mikrorstitev* pa lahko hitro privede do t.i. **Nano-service antivzorca**
 - preveč drobno razdeljene funkcionalnosti,
 - več omrežne komunikacije in večja latenca,
 - večja operativna obremenitev (nameščanje, nadzor, konfiguracija),
 - slabša preglednost in težje razumevanje delovanja sistema.
- Cilj je najti **ustrezno granulacijo**
 - storitve naj bodo dovolj majhne za neodvisno delo ekipe, a dovolj velike, da imajo **smiselno poslovno odgovornost oz. zmožnost**

19

Kako namestiti veliko število mikrorstitev?

Monolit

- 1 aplikacija
- 1 deployment
- 1 konfiguracija
- enotno spremljanje

Mikrorstitev

- N storitev (10, 100, 1000 ...)
- N deploymentov
- N konfiguracij
- N logov/metrik/sledenje
- razpoložljivost + skaliranje po storitvah

20

Kako namestiti veliko število mikrororitev?

• Operativni izzivi pri mikrororitvah:

- **CI/CD**: nameščanje in posodabljanje storitev
 - **Konfiguracija**: okolja, skrivnosti (secrets), parametri
 - **Opazljivost**: monitoring, logi, sledenje (tracing)
 - **Zanesljivost**: razpoložljivost, toleranca na napake
 - **Skaliranje**: horizontalno, funkcijsko (po storitvah)
- Mikrororitve povečajo fleksibilnost razvoja, vendar zahtevajo **močno operativno podporo (platformo/orkestracijo)**.

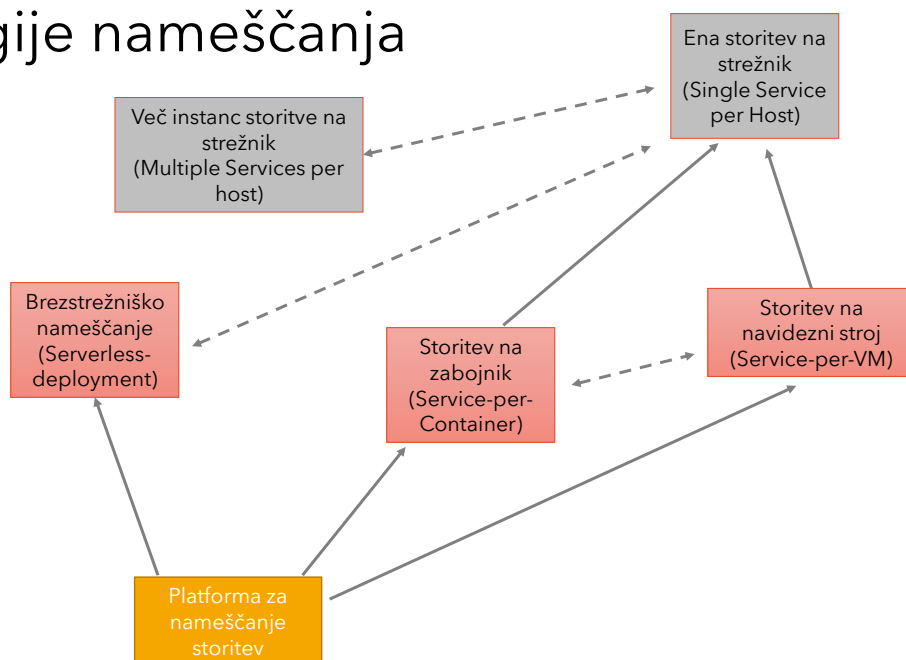
21

Izzivi, povezani z nameščanjem in upravljanjem storitev

Heterogenost tehnologij	• storitve so lahko razvite v različnih jezikih, ogrodjih in celo različnih različicah platform.
Več instanc na storitev	• zaradi prepustnosti in razpoložljivosti se vsaka storitev pogosto izvaja v več instancah.
Hitro in pogosto uvajanje	• razvoj in nameščanje morata biti učinkovita; nujna je uvedba neprekinjene integracije in dostave (CI/CD) .
Kratek čas do produkcije	• cilj je minimalen čas od izgradnje do uvedbe in zagona storitve v produkcijskem okolju.
Neodvisno uvajanje in skaliranje	• storitve morajo biti nameščene ter skalirane ločeno, brez vpliva na druge.
Izolacija instanc in omejevanje virov	• instance morajo biti izolirane, poraba virov pa nadzorovana (npr. CPU, pomnilnik).
Zanesljivost in stroškovna učinkovitost	uvajanje mora biti ponovljivo, varno in cenovno učinkovito – tudi večkrat na dan .

22

Strategije nameščanja



Prirejeno po: <https://microservices.io/i/MicroservicePatternLanguage.jpg>

23

Vzorec: Platforma za nameščanje storitev

- Vzorec temelji na **uporabi platforme za uvajanje (deployment platform)**, ki zagotavlja infrastrukturo za avtomatizirano uvajanje in upravljanje storitev.
- **Platforma nudi abstrakcijo izvajalnega okolja in poskrbi za**
 - razporejanje instanc,
 - nadzor stanja (health checks),
 - ponovni zagon ob napaki,
 - uravnoteženje obremenitve in
 - samodejno skaliranje.
- Cilj je **višja razpoložljivost in** bolj predvidljivo obratovanje, brez ročnega upravljanja posameznih instanc.

24

Primeri platform

1. Orkestracija kontejnerjev (najpogostejše pri mikrorstoritvah)

- **Kubernetes** (<https://kubernetes.io/>)
- **Docker Swarm** (<https://docs.docker.com/engine/swarm>)
- **HashiCorp Nomad** (<https://developer.hashicorp.com/nomad/docs>)

2. Upravljana orkestracija v oblaku

- **Amazon ECS** (<https://docs.aws.amazon.com/ecs/>)
- **Amazon EKS** (managed Kubernetes) (<https://docs.aws.amazon.com/eks>)
- **Azure Kubernetes Service (AKS)** (<https://learn.microsoft.com/en-us/azure/aks>)
- **Google Kubernetes Engine (GKE)** (<https://docs.cloud.google.com/kubernetes-engine/docs>)

3. "Serverless containers" (kontejnerji brez upravljanja strežnikov)

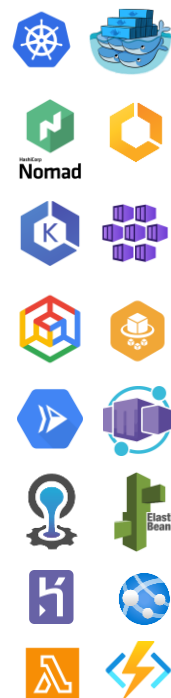
- **AWS Fargate** (<https://aws.amazon.com/documentation-overview/fargate>)
- **Google Cloud Run** (<https://docs.cloud.google.com/run/docs>)
- **Azure Container Apps** (<https://learn.microsoft.com/en-us/azure/container-apps>)

4. PaaS (platforma za aplikacije, manj skrbi za infrastrukturo)

- **Cloud Foundry** (<https://docs.cloudfoundry.org/>)
- **AWS Elastic Beanstalk** (<https://docs.aws.amazon.com/elastic-beanstalk/>)
- **Heroku** (<https://devcenter.heroku.com/articles/how-heroku-works>)
- **Azure App Service** (<https://learn.microsoft.com/en-us/azure/app-service>)
- **Google App Engine** (<https://docs.cloud.google.com/appengine/docs>)

5. Serverless / Functions (FaaS)

- **AWS Lambda** (<https://docs.aws.amazon.com/lambda/>)
- **Azure Functions** (<https://learn.microsoft.com/en-us/azure/azure-functions>)
- **Google Cloud Run Functions** (<https://docs.cloud.google.com/functions/docs>)
- **Cloudflare Workers** (edge funkcije) (<https://developers.cloudflare.com/workers/>)
- **Netlify Functions** (<https://docs.netlify.com/build/functions/overview>)
- **Vercel Functions** (<https://vercel.com/docs/functions>)



25

Vzorec: Več instanc storitve na strežnik

Strežnik (fizični ali VM)

Storitev A
Instanca 1Storitev B
Instanca 1Storitev C
Instanca 1

Strežnik (fizični ali VM)

Storitev A
Instanca 2Storitev B
Instanca 2Storitev C
Instanca 2

Klasičen pristop, kjer na enem strežniku (fizičnem ali navideznem - VM) izvajamo več instanc različnih storitev.

Instance se lahko izvajajo:

- v **ločeni izolaciji** (npr. ločeni procesi ali zabojniki), ali
- v **skupnem izvajalnem okolju** (npr. isti izvajalni strežnik / application server / runtime), kjer več storitev deli isto okolje izvajanja.

Prirejeno po: <https://www.dotnettricks.com/learn/microservices/deployment-patterns>

26

Vzorec: Več instanc storitve na strežnik

Prednosti

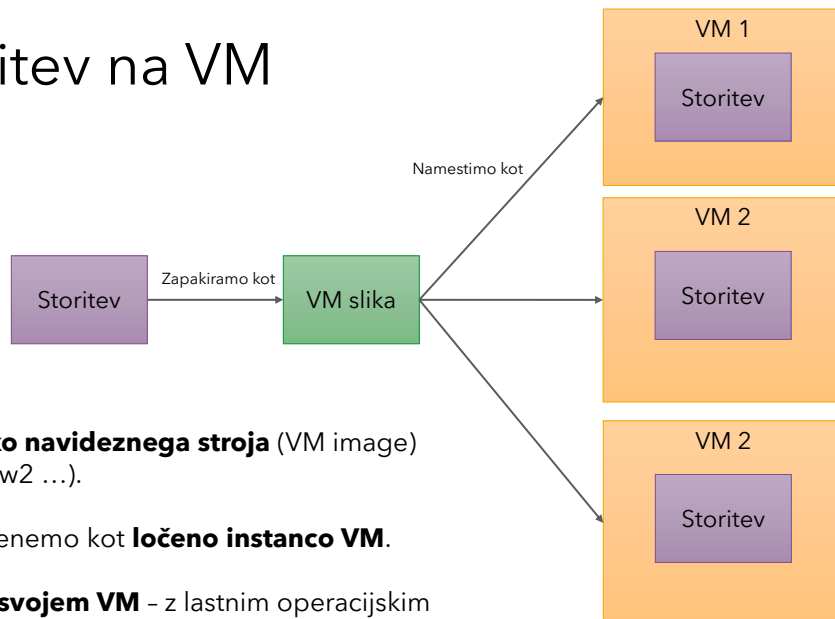
- **Učinkovita raba virov:** več instanc si deli isti strežnik (CPU, pomnilnik, disk), zato potrebujemo manj infrastrukture.
- **Relativno enostavna namestitve (v manjšem obsegu):** storitve lahko namestimo neposredno na strežnik, brez orkestracije; pri večjem številu storitev pa se operativna kompleksnost hitro poveča.

Slabosti

- **Slabša izolacija:** težave ene storitve lahko vplivajo na druge (npr. poraba pomnilnika, obremenitev CPU).
- **Omejen nadzor virov:** brez namenskih mehanizmov (npr. omejitve na ravni OS/zabojnikov) je težje natančno določiti in uveljaviti porabo CPU/pomnilnika po storitvi.
- **Konflikti odvisnosti:** če storitve delijo isto izvajalno okolje ali sistemske knjižnice, lahko pride do težav z različicami (»dependency hell«).
- **Slabša enkapsulacija tehnologij:** storitve so bolj vezane na skupno okolje in konfiguracijo strežnika.

27

Vzorec: Storitve na VM



Storitev pripravimo kot **sliko navideznega stroja** (VM image) (npr. AMI, VHD/VMDK, qcow2 ...).

Vsako instanco storitve zaženemo kot **ločeno instanco VM**.

➡ **Vsaka instanca teče v svojem VM** - z lastnim operacijskim sistemom in izoliranim izvajalnim okoljem.

Prirejeno po: <https://www.dotnettricks.com/learn/microservices/deployment-patterns>

28

Vzorec: Storitve na VM

Prednosti:

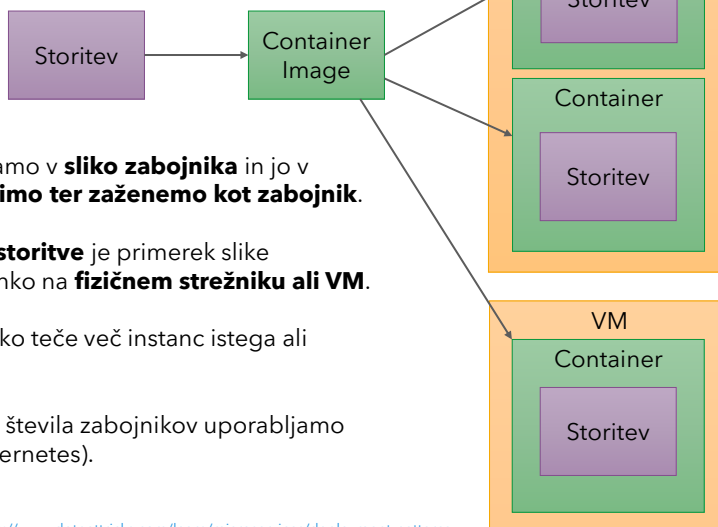
- **Visoka izolacija** med storitvami (ločeni VM-ji zmanjšajo medsebojni vpliv).
- **Boljše upravljanje:** Zrela infrastruktura in orodja: virtualizacijske/oblačne platforme ponujajo preverjene mehanizme za nadzor, posodabljanje in upravljanje VM.
- **Enkapsulacija tehnologij:** VM vključuje celotno izvajalno okolje (OS + runtime + odvisnosti), kar zmanjša konflikte in poveča ponovljivost.
- Dobro se poveže z oblakom: možna sta **samodejno skaliranje in uravnoteženje obremenitve** z oblačnimi storitvami.

Slabosti:

- **Slabša izkoriščenost virov** (vsak VM vključuje svoj OS) → praviloma višji stroški.
- **Počasnejša izgradnja slik** (večja količina podatkov za prenos in shranjevanje).
- **Počasnejši zagon** v primerjavi z lažjimi pristopi (npr. zabojniki).
- **Več operativnega dela pri velikem številu storitev:** več VM-jev pomeni več patchanja, image lifecycle, konfiguracij, če ni dobre avtomatizacije (IaC).

29

Vzorec: Storitve na zabojnik



Vsako storitev zapakiramo v **sliko zabojnika** in jo v ciljnem okolju **namestimo ter zaženemo kot zabojnik**.

Posamezna instanca storitve je primerek slike zabojnika - izvaja se lahko na **fizičnem strežniku ali VM**.

Na istem gostitelju lahko teče več instanc istega ali različnih zabojnikov.

Za upravljanje večjega števila zabojnikov uporabljamo **orkestracijo** (npr. Kubernetes).

Med najbolj popularne platforme zagotovo sodi Docker

Na voljo so številne rešitve za upravljanje Docker zabojnikov druge rešitve, npr.

- Kubernetes
- Marathon/ Mesos
- Amazon EC2
- ...

Prereizeno po: <https://www.dotnettricks.com/learn/microservices/deployment-patterns>

30

Vzorec: Storitve na zabojnik

Prednosti:

- **Dobra izolacija** - storitve tečejo ločeno, z manj medsebojnega vpliva.
- **Enostavnejše upravljanje** - standardiziran način pakiranja, zagona, ustavljanja in nadzora instanc.
- **Enkapsulacija tehnologije** - zabojnik vključuje vse odvisnosti; po zagonu ni pomembno, ali je storitev napisana v Ruby, Node.js ipd. - ključna je slika zabojnika.
- **Učinkovita raba virov** - manjši režijski stroški kot pri VM, več instanc na istem gostitelju.
- **Hitro uvajanje** - slike se hitro prenašajo, storitve se praviloma hitro zaženejo.

Slabosti:

- **Kompleksnost pri večjem številu storitev:** pogosto potrebujemo orkestracijo (npr. Kubernetes) za razporejanje, nadzor in skaliranje.
- **Slabša izolacija kot pri VM:** zabojniki delijo jedro OS, zato je izolacija praviloma manj stroga kot pri VM.
- **Kompleksnejše odkrivanje storitev:** service discovery, load balancing.
- **Zahtevnejša opazljivost in diagnostika:** logi/metrike/sledenje so porazdeljeni med veliko instanc.
- **Upravljanje konfiguracij, skrivnosti in stanja:** konfiguracije po okoljih, "secrets" ter trajno shranjevanje zahtevajo dodatne mehanizme.

31

Docker-compose poenostavlja razvoj (v ekipah)

Tipična storitev poleg kode pogosto potrebuje tudi podatkovno bazo, sporočilni sistem in druge odvisnosti.

V ekipah je velik izziv zagotoviti, da imajo vsi razvijalci enako in pravilno različico izvajalnega okolja ter infrastrukturnih komponent (knjižnice, baze, posredniki sporočil ...), še posebej pri sodobnih sistemih z več storitvami.

Docker Compose omogoča, da celotno razvojno okolje opišemo kot konfiguracijo in ga na vsakem računalniku zaženemo na enak način (»en ukaz« → enako okolje).

32

Vzorec: brezstrežniško nameščanje

- Namen je **skriti podrobnosti infrastrukture**
 - ponudnik oblaka prevzame upravljanje strežnikov, skaliranja in razpoložljivosti.
 - Razvijalci se osredotočijo na kodo in konfiguracijo, ne na upravljanje izvajalnega okolja.
- Glavni obliki:
 - **Function as a Service (FaaS)** – izvajanje posameznih funkcij na zahtevo (dogodki/HTTP).
 - Primeri: AWS Lambda, Azure Functions, Google Cloud Functions, Apache OpenWhisk.
 - **Backend as a Service (BaaS)** – pripravljene oblačne zaledne storitve (npr. avtentikacija, shranjevanje, obveščanje).
 - Primeri: Firebase, Supabase, (tudi upravljane baze in shrambe kot del zalednih storitev: npr. DynamoDB, Cloud Storage).
- Prednosti
 - praviloma samodejno skaliranje,
 - večja prožnost in
 - pogosto nižji stroški (plačilo po uporabi).

33

Komunikacijski vzorci v mikrostoritvenih rešitvah

Pri mikrostoritvah gradimo **porazdeljen sistem**, kjer se **storitve izvajajo v ločenih procesih**.

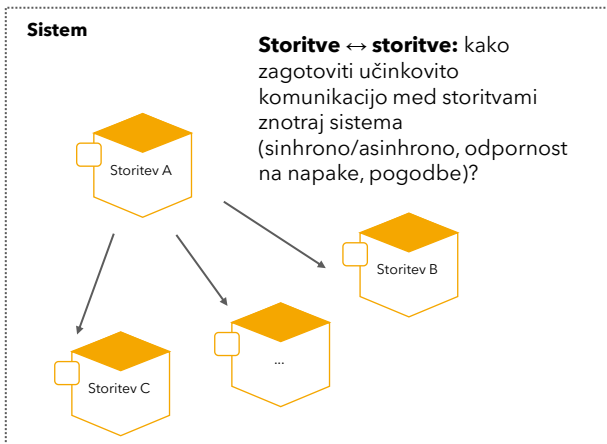
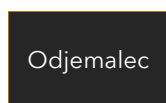
To prinese izzive pri načrtovanju in izvedbi **medprocesne komunikacije** med storitvami
- izbira protokolov, zanesljivost, latenca, obravnava napak, ...

Zato so potrebni ustrezni **komunikacijski vzorci**, ki pomagajo zagotoviti učinkovito in robustno povezovanje storitev.

34

Izzivi pri komunikaciji v mikrostoritvah

1. Odjemalci ↔ storitve: kako zagotoviti učinkovito interakcijo med odjemalci sistema in storitvami (usmerjanje, avtentikacija, enotna vstopna točka, verzioniranje API)?

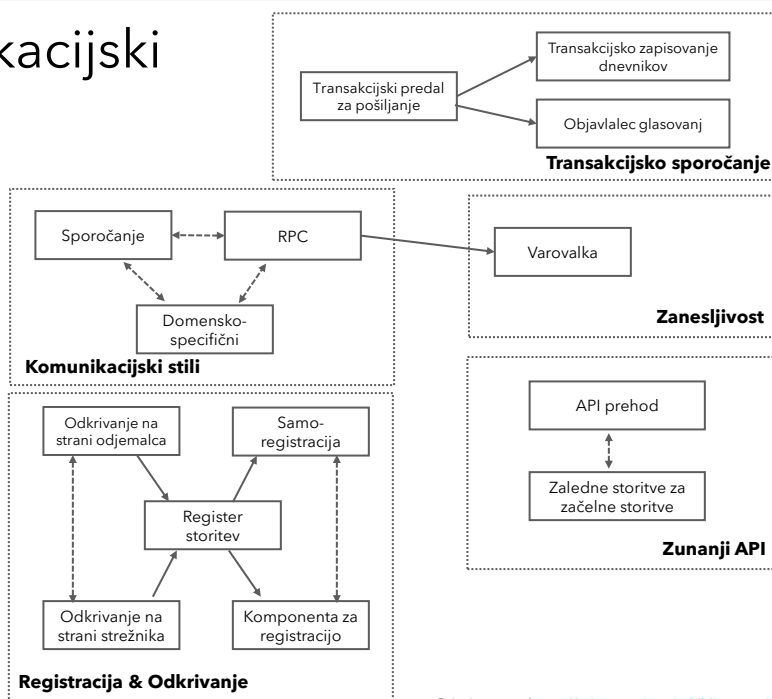


V mikrostoritveni arhitekturi je lahko tudi **več sto končnih točk (endpoints)**.

Zato je ključno vprašanje: **kako naj storitve zanesljivo in učinkovito najdejo ter kličejo druge storitve** (odkrivanje storitev, uravnoteženje obremenitve, časovne omejitve, ponovni poskusi)?

35

Komunikacijski vzorci



Prirejeno po: <https://microservices.io/i/MicroservicePatternLanguage.jpg>

36

Komunikacijski vzorci v mikrostoritvah

- **API prehod (API Gateway):** enotna vstopna točka v sistem, ki sprejme zahtevo in jo usmeri do ustrezne mikrostoritve
 - po potrebi doda tudi avtentikacijo, omejevanje prometa, ...
- **Komunikacijski stili:**
 - **sinhroni slog** (npr. RPC/HTTP, REST, gRPC),
 - **asinhroni slog** (sporočanje prek posrednika, dogodki).
- **Odkrivanje storitev (Service Discovery):**
 - v sodobnih okoljih (oblak, zabojniki) **naslovi storitev niso statični**,
 - pri velikem številu storitev statična konfiguracija naslovov in vrat ni obvladljiva,
 - zato potrebujemo **dinamično odkrivanje storitev** (registracija storitev in iskanje po imenih) – za to obstajajo uveljavljeni vzorci.

37

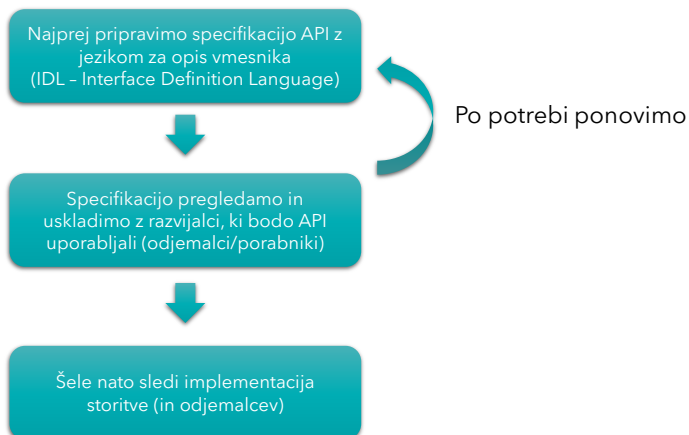
API-first pristop k zasnovi aplikacijskih programskih vmesnikov

- Pri mikrostoritvah je ključno, da **API-je načrtujemo pred implementacijo**, saj so vmesniki temelj povezovanja med storitvami.
- Pri REST pristopu je API pogosto nastal kot »stranski produkt« implementacije, zato se tehnične pogodbe v praksi dolgo niso formalno opredeljevale.
 - Sčasoma se je pokazala potreba po **specifikaciji API** (tehnični pogodbi), ki je neodvisna od implementacije.
 - Zato se uporabljajo standardi in orodja, kot je **OpenAPI (Swagger)**, ki omogočajo:
 - jasno definicijo končnih točk, parametrov in odzivov,
 - lažje usklajevanje med ekipami,
 - generiranje dokumentacije in odjemalcev.

38

API-first pristop razvoja

API = pogodba med odjemalcem in storitvijo



Primeri jezikov IDL:



39

Formati sporočil v APIjih

Pri zasnovi API-ja določimo tudi **format sporočil**, ki se prenašajo med odjemalcem in storitvijo.

Besedilni formati (tekstovni)

- npr. **JSON, XML**
- sporočila so **berljiva človeku** in pogosto **samoopisna**, kar olajša razvoj, razhroščevanje in uporabo API-ja
- slabost: večja količina podatkov in praviloma **počasnejše razčlenjevanje (parsing)**

Binarni formati

- npr. **Protocol Buffers, Avro**
- sporočila so bolj **kompaktna** in omogočajo hitrejšo **serializacijo/deserializacijo**
- slabost: pogosto niso samoopisna, zato je potrebna **strožja shema**, kar lahko oteži uporabo in upočasni razvoj (če ni dobre podpore z orodji)

40

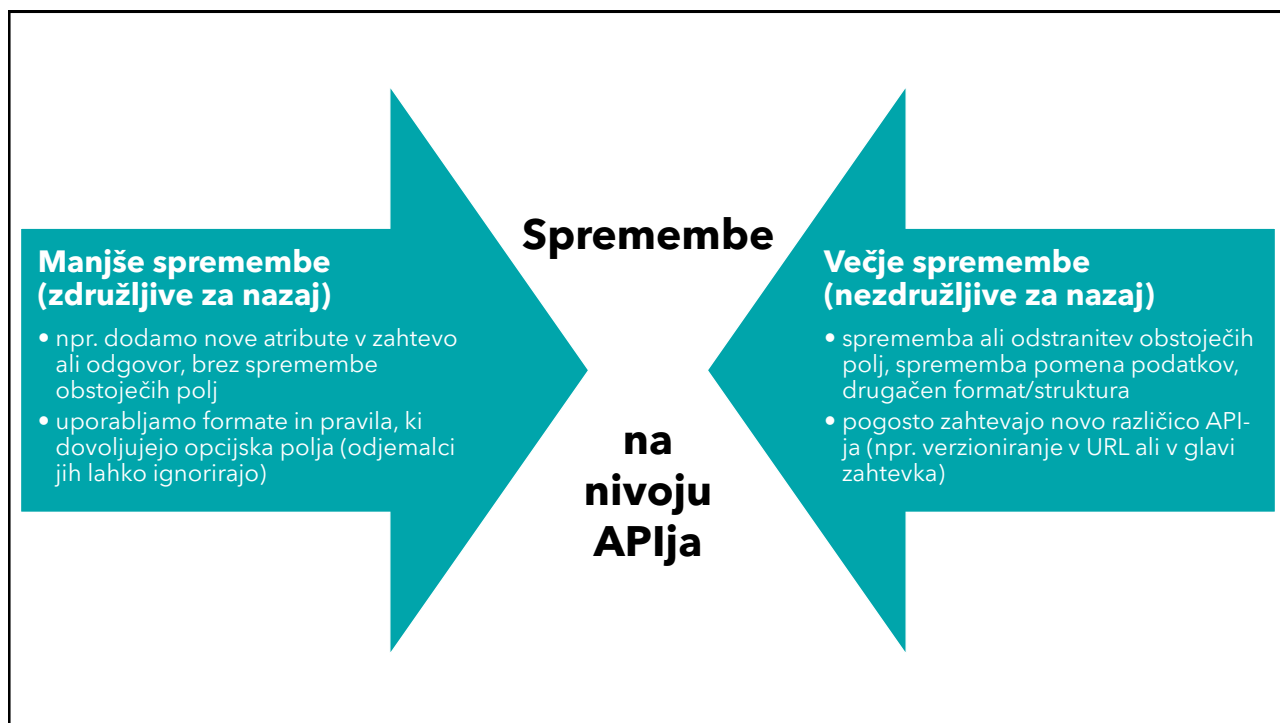
Spremembe v APIjih

V poslovnih informacijskih rešitvah so spremembe **neizogibne**.

Spremembe se pojavljajo na vseh ravneh sistema – tudi na nivoju **aplikacijskih programskih vmesnikov (API)**.

V mikrororitvenih sistemih so spremembe pogosto **pogostejše**, ker se storitve razvijajo in uvajajo neodvisno ter v krajših iteracijah.

41



42

Vzorec: **API prehod** (API Gateway)

- API prehod **rešuje izzive, povezane z zagotavljanjem učinkovite komunikacije odjemalcev z mikrostoritvami**, še posebej ko:
 - so API-ji storitev **drobnozrnat**, zato mora odjemalec klicati **več storitev** za en uporabniški primer,
 - različni odjemalci (npr. splet, mobilna aplikacija) potrebujejo **različne podatke**,
 - se število instanc in njihove **lokacije** (naslov + vrata) dinamično spreminjajo,
 - se notranja razdelitev sistema sčasoma spreminja (delitev storitev, dodajanje novih) – odjemalci ne bi smeli poznati **notranje strukture** mikrostoritev,
 - storitve uporabljajo različne **protokole**, ki niso nujno optimalni za neposredno uporabo prek spleta.
- API prehod zato deluje kot **enotna vstopna točka**, ki zahteve usmeri, po potrebi združi odgovore in skrije notranjo kompleksnost sistema.

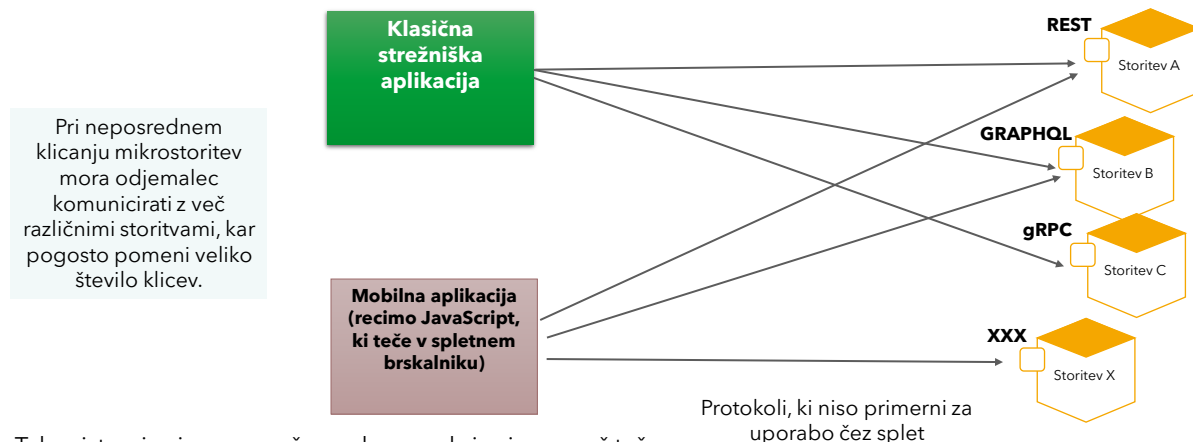
43

Vzorec: **API prehod** (API Gateway)

- Namesto enega »univerzalnega« API-ja lahko prek API prehoda izpostavimo **vmesnike, prilagojene posameznim odjemalcem** (npr. splet, mobilna aplikacija), brez potrebe po spremembah v mikrostoritvah.
- API prehod lahko prevzame tudi **prečne (skupne) funkcionalnosti**, na primer:
 - **varnost**: avtentikacija in avtorizacija klicev,
 - **usmerjanje** in združevanje rezultatov več storitev,
 - **omejevanje prometa** (rate limiting) in
 - **beleženje** (logging).

44

Vzorec: Direktno povezovanje front-end rešitev z zalednimi (back-end) storitvi



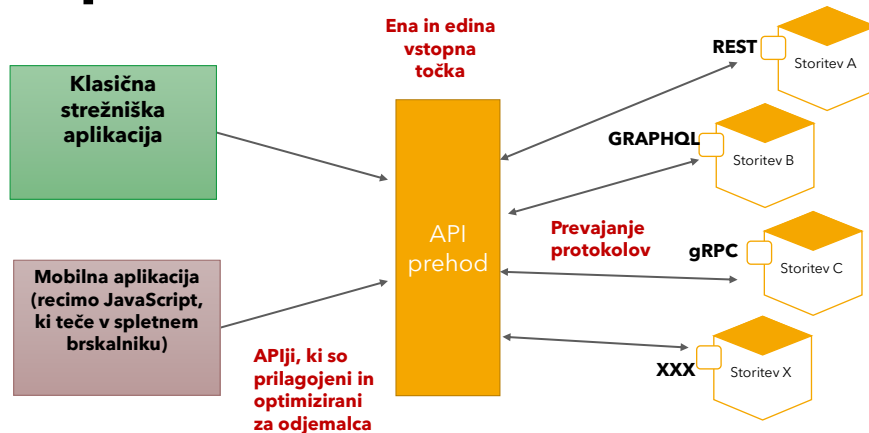
Tak pristop je sicer mogoč, vendar v praksi prinese več težav:

- **večja količina omrežnih klicev** in s tem **večja latenca**,
- **razlike v protokolih** in oblikah API-jev,
- **upravljanje naslovov, vrat in dinamičnih lokacij storitev**,
- **večja odvisnost od notranje strukture sistema** (spremembe storitev hitro vplivajo na odjemalca).

Prilagojeno po: <https://microservices.io/patterns/apigateway.html>

45

Vzorec: **API prehod**



API prehod lahko zahteve odjemalca

- **posreduje** ali **preusmeri** do ustrezne mikrostoritve,
- **razširi/orkestrira** klic čez več povezanih mikrostoritev (npr. za sestavo enotnega odgovora),
- izpostavi **API, prilagojen odjemalcu** (npr. različni vmesniki za spletno in mobilno aplikacijo),
- po potrebi **doda še skupne funkcionalnosti** (npr. varnost, beleženje, omejevanje prometa).

Prilagojeno po: <https://microservices.io/patterns/apigateway.html>

46

API prehod: prednosti

- Odjemalcem ni treba izvajati **odkrivanja storitev** (naslovi, vrata, dinamične lokacije so skrite za preходом).
- Omogoča **API, prilagojen posameznemu odjemalcu** (npr. splet, mobilna aplikacija), in optimiziran za uporabo prek omrežja.
- Običajno je potrebnih **manj zahtevkov** in **manj prenosa podatkov**, saj prehod lahko združi podatke iz več storitev v en odgovor.
- S tem se pogosto zmanjša **zakasnitev** na strani odjemalca (manj omrežnih skokov).
- **Poenostavi kodo odjemalca**: logiko za zaporedne/vzporedne klice ter združevanje rezultatov lahko prevzame API prehod.
- Omogoča tudi **prevajanje med protokoli** (npr. notranji gRPC/sporočila ↔ zunanji HTTP/REST), kadar notranji protokoli niso primerni za neposredno spletno uporabo.

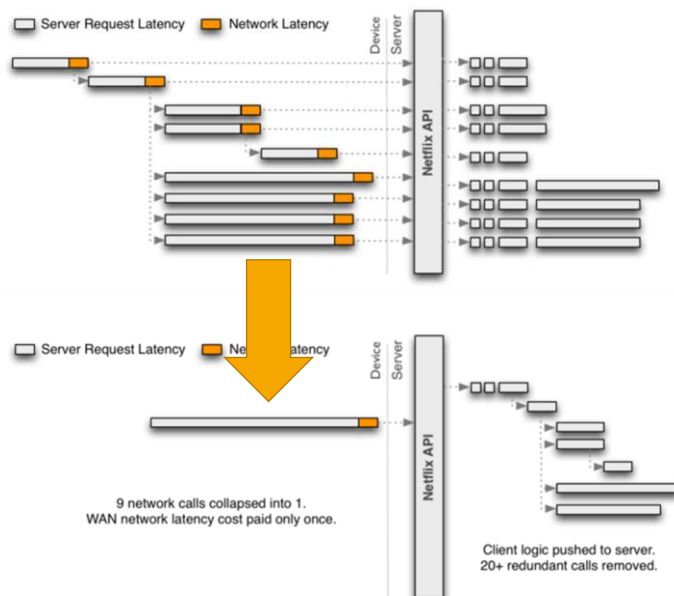
47

API prehod: izzivi

- **Dodatna kompleksnost**: API prehod je nova komponenta, ki jo je treba **razviti, uvajati in upravljati**.
- **Visoka razpoložljivost je nujna**: prehod postane kritična vstopna točka, zato mora biti dobro **skaliran, nadzorovan in odporen na napake**.
- **Potencialno daljši odzivni čas**: zaradi dodatnega omrežnega "skoka" se lahko skupni odziv nekoliko poveča (čeprav se to pogosto izravna z manj klici proti storitvam).

48

Primer: Netflix API



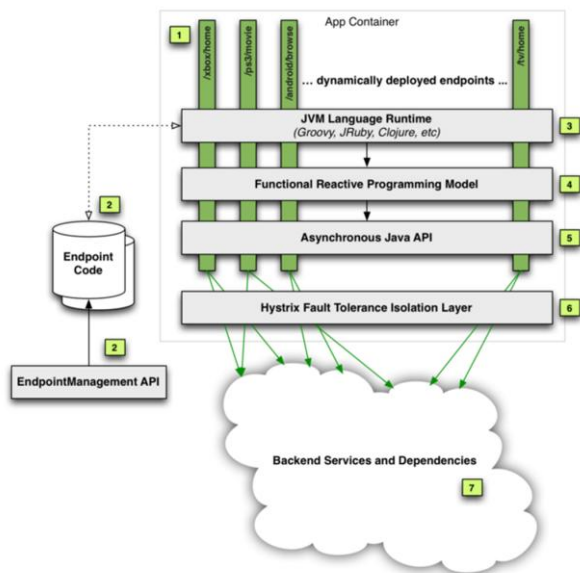
Optimizirali so št. klicev odjemalca

Manjše št. klicev iz strani odjemalca → manjša zakasnitev, saj je ta vezana samo na en klic in ne na ločene klice, ki so se izvajali prej

Vir: <https://netflixtechblog.com/optimizing-the-netflix-api-5c9ac715cf19>

49

Primer: Netflix API



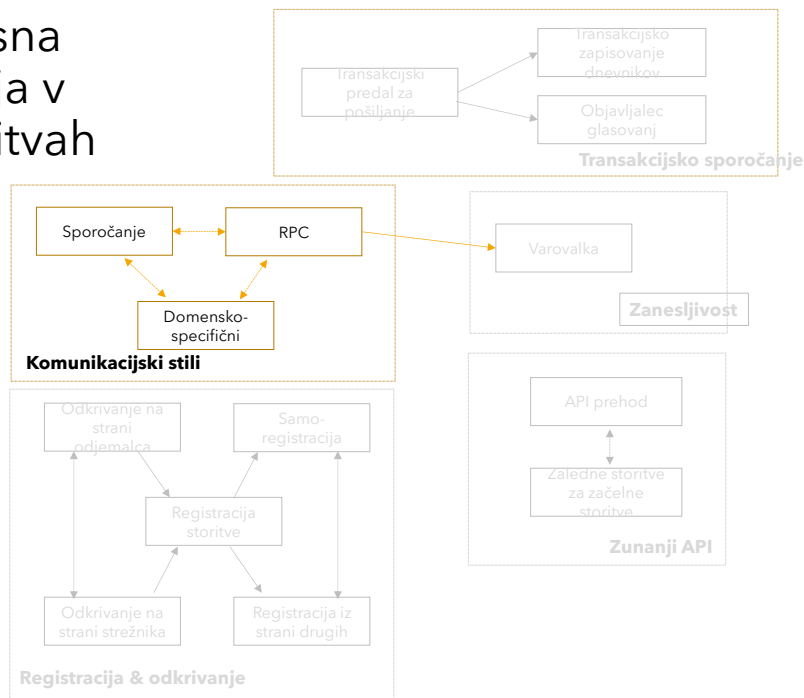
Pri podjetju Netflix so zasnovali arhitekturo, ki omogoča:

- Dinamično dodajanje končnih točk
- Izvajalno okolje za izvajanje storitev, ki so spisane v različnih programskih jezikih
- Asinhroni in reaktivni programski model
- Nivo storitev za upravljanje z napakami
- Abstrakcijo dostopa do zalednih storitev in odvisnosti

Vir: <https://netflixtechblog.com/optimizing-the-netflix-api-5c9ac715cf19>

50

Med-procesna komunikacija v mikrost. rešitvah



51

Zakaj med-procesna komunikacija?

- **Monolitne aplikacije** se tipično izvajajo znotraj **enega procesa**. Klici metod/procedur so **lokalni** in se izvajajo neposredno v programskem jeziku aplikacije.
- **Mikrostoritvene aplikacije** so drugačne: storitve se izvajajo kot **ločeni procesi** (pogosto na različnih strežnikih ali v zabojnikih).
- Zato storitve ne morejo komunicirati z lokalnimi klici, temveč potrebujejo mehanizme za **medprocesno komunikacijo** (npr. HTTP/REST, gRPC, sporočilni sistemi).

52

Vzorci komuniciranja v mikrostoritvah

- V mikrostoritvenih rešitvah najpogosteje izbiramo med temi slogi komunikacije:
 - **RPC (sinhrona komunikacija)**
 - neposreden klic oddaljene storitve in čakanje na odgovor
 - npr. REST/HTTP, gRPC, ...
 - **Sporočanje (asinhrona komunikacija)**
 - izmenjava sporočil ali dogodkov prek posrednika;
 - pošiljatelj praviloma ne čaka na takojšen odgovor.
 - **Domensko specifični protokoli/stili**
 - komunikacija, prilagojena domeni
 - npr. **dogodkovno usmerjena arhitektura**, objava-naročanje, procesi tipa saga

53

Možni načini interakcije

Topologija	Sinhrono (zahteva-odgovor)	Asinhrono (sporočila/dogodki)
1 : 1	Zahteva-odgovor (HTTP/gRPC)	Ukaz-odgovor prek sporočil (async request-reply, korelacijski ID)
1 : N	Fan-out klici (pogosto problematično: verige klicev, "cascading failures")	Objavi-naroči (Pub/Sub): dogodek → več naročnikov (npr. "OrderCreated")

Dogodek = enosmerno obvestilo, da se je nekaj zgodilo (fire-and-forget).
 Ukaz = zahteva, da nekdo nekaj naredi (lahko sinhrono ali asinhrono).

54

Med-procesna komunikacija na osnovi **RPC**



- Med-procesno komunikacijo v osnovi lahko implementiramo na osnovi izbranega sistema RPC
 - Odjemalec na osnovi nadomestne kode proži metodo na strežniški strani, strežnik pa preko povezovalnih komponent odgovor na zahtevo
 - Povezovalne komponente na osnovi izbrane tehnologije abstrahirajo vso logiko, ki je potrebna za izgradnjo sporočil in prenos sporočil preko komunikacijskega kanala
- Na voljo je veliko sistemov oz. tehnologij tipa RPC
 - Največkrat se uporabljajo rešitve na osnovi **REST**
 - Na voljo so pa novejši protokoli in standardi, ki omogočajo implementacijo takšne komunikacije (**gRPC**, **GraphQL**, itd.)

55

RPC: prednosti in slabosti

Prednosti

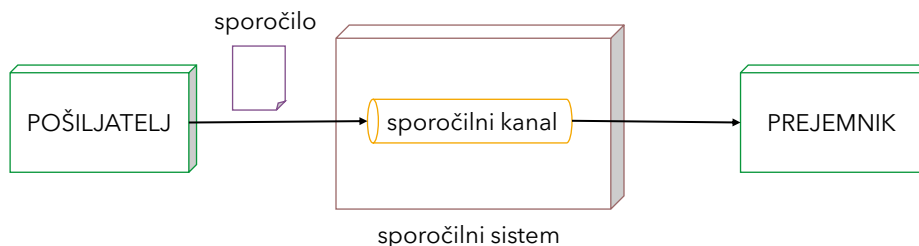
- **Enostaven in razvijalcem poznan pristop.**
- Naravno podpira vzorec **zahteva/odgovor** (request-response).
- Ne zahteva dodatnih posrednih komponent (npr. **sporočilnega sistema**).

Slabosti

- Večinoma omejen na **sinhrono** komunikacijo tipa zahteva/odgovor.
- Storitve mora biti ob klicu **dosegljiva**; v nasprotnem primeru klic spodleti ali se izteče (timeout).
- Odjemalec mora poznati **način naslavljanja storitve** (lokacija instance ali mehanizem odkrivanja storitev).

56

Med-procesna komunikacija na osnovi **sporočanja**



- Pri **asinhroni komunikaciji** storitve komunicirajo prek **sporočilnega sistema** (message broker).
- **Pošiljatelj** sporočilo odda v sporočilni sistem, ne da bi moral čakati na takojšen odgovor.
- **Prejemnik** (ali več prejemnikov) sporočilo prevzame iz sporočilnega sistema in ga obdelava, ko je na voljo.
- Najpogostejša mehanizma:
 - **vrste (queues)** – sporočilo obdelava en prejemnik,
 - **teme (topics)** – paradigma **objavi/naroči (publish/subscribe)**, kjer sporočilo prejme več naročnikov.

57

Sporočanje: prednosti in slabosti

Prednosti

- **Šibka sklopljenost** med odjemalci in storitvami (neodvisnost v času in prostoru).
- **Obstojnost sporočil** in bolj zanesljiva dostava: sporočilni sistem lahko sporočila **trajnostno hrani** in jih odjemalcem dostavi, ko so na voljo.
- Omogoča različne komunikacijske vzorce (npr. **vrste, objavi/naroči**, dogodkovno obveščanje).

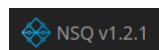
Slabosti

- Sporočilni sistem prinese **dodatno infrastrukturo** in s tem večjo kompleksnost (upravljanje, nadzor, razpoložljivost).
- **Zahteva/odgovor** je težje implementirati (potrebna korelacija sporočil, asinhron potek).
- Odjemalci morajo poznati in pravilno konfigurirati dostop do **sporočilnega sistema** (naslov, varnost, pravila povezave).

58

Sporočilni sistemi

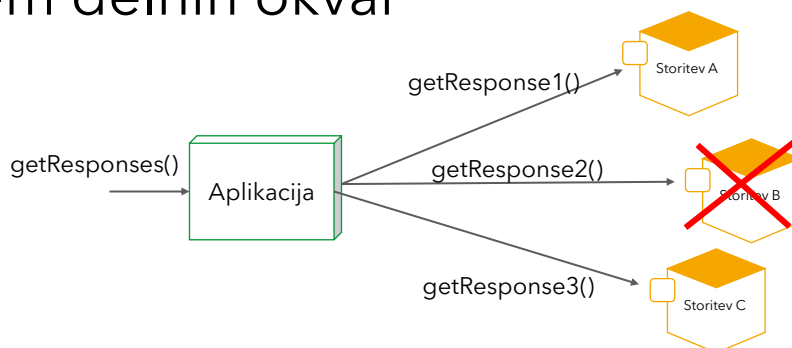
- Memphis (<https://memphis.dev/>)
- RabbitMQ (<https://www.rabbitmq.com/>)
- ActiveMQ (<https://activemq.apache.org/>)
- Apache Kafka (<https://kafka.apache.org/>)
- NSQ (<https://nsq.io/>)
- WSO2 Message Broker (<https://wso2.com/products/message-broker/>)
- ZeroMQ (<https://zeromq.org/>)
- IBM MQ (<https://www.ibm.com/products/mq>)
- Amazon SQS (<https://aws.amazon.com/sqs/>)
- Google Cloud Pub/Sub (<https://cloud.google.com/pubsub>)
- ...



Cloud Pub/Sub

59

Problem delnih okvar



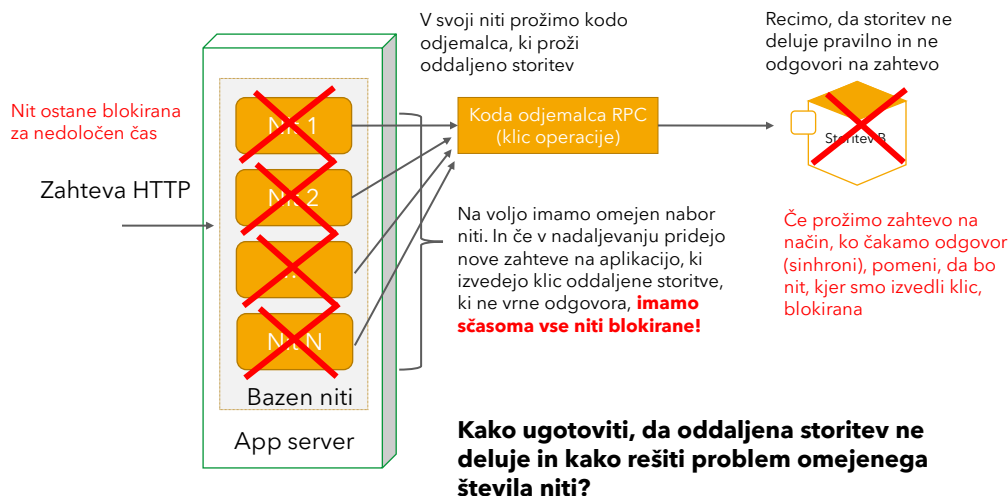
Pri gradnji storitev želimo, da so le-te visoko skalabilne, dosegljive

Vedno obstaja nevarnost, da bo določena storitev prenehala delovati, ali bo začela delovati zelo počasi ali se obnašati izven pričakovanih okvirjev.

Kaj se zgodi, ko pri izvajanju klicev ločenih storitev ena storitev preneha delovati?

60

Problem: omejeno število niti



61

Kako rešiti problem z omejenim naborom niti?

- **Časovne omejitve (timeout):** za omrežne klice določimo največji čas čakanja, da niti ne ostanejo blokirane predolgo.
- **Omejevanje sočasnih klicev:** oddaljene klice izvajamo prek **omejenega bazena niti**.
 - Če so vse niti zasedene in se čakalna vrsta polni, to pogosto pomeni, da je odvisna storitev počasna ali nedosegljiva.
- Vzorec **prekinjevalec tokokroga (Circuit Breaker):**
 - spremljamo stopnjo napak (npr. več zaporednih neuspešnih klicev),
 - ob prekoračitvi praga »odpremo varovalko« in **začasno blokiramo nove klice** do te storitve,
 - po določenem času poskusimo znova; če klic uspe, se sistem vrne v normalno stanje, sicer varovalko ponovno aktiviramo.

62

Kako ravnati ob napaki pri klicu storitve?

- Ko klic oddaljene storitve ne uspe, imamo več možnosti odziva:
 - **Vrnemo privzeti (nadomestni) odziv** (*fallback*), da odjemalec vseeno dobi smiseln rezultat.
 - **Vrnemo podatke iz medpomnilnika (cache)**, če je sprejemljivo, da so podatki morda nekoliko zastareli.
 - **Vrnemo informacijo o napaki** (ustrezna koda in sporočilo), kadar nadomestni odziv ni smiseln ali bi bil zavajajoč.
- Uporabimo ogrodja/knjižnice za **odpornost na napake**
 - npr. timeouts,
 - ponovni poskusi,
 - circuit breaker,
 - omejevanje sočasnih klicev
 - ...

63

Orodja za zagotavljanje odpornosti na napake v delovanju

- **Hystrix** (<https://github.com/Netflix/Hystrix>) - knjižnica, ki so jo razvili pri podjetju Netflix za nadzor interakcij med porazdeljenimi storitvami.
- **Spring Cloud Hystrix** (<https://spring.io/guides/gs/cloud-circuit-breaker/>)
- **Failsafe** (<https://failsafe.dev>) - light-weight knjižnica za implementacijo rešitev, ki omogočajo odpornosti na napake
- **Resilience4J** (<https://resilience4j.readme.io>) - light-weight ogrodje za implementacijo vzorcev odpornosti na napake. Ogradje podobno Hystrix.
- ...



HYSTRIX
DEFEND YOUR APP



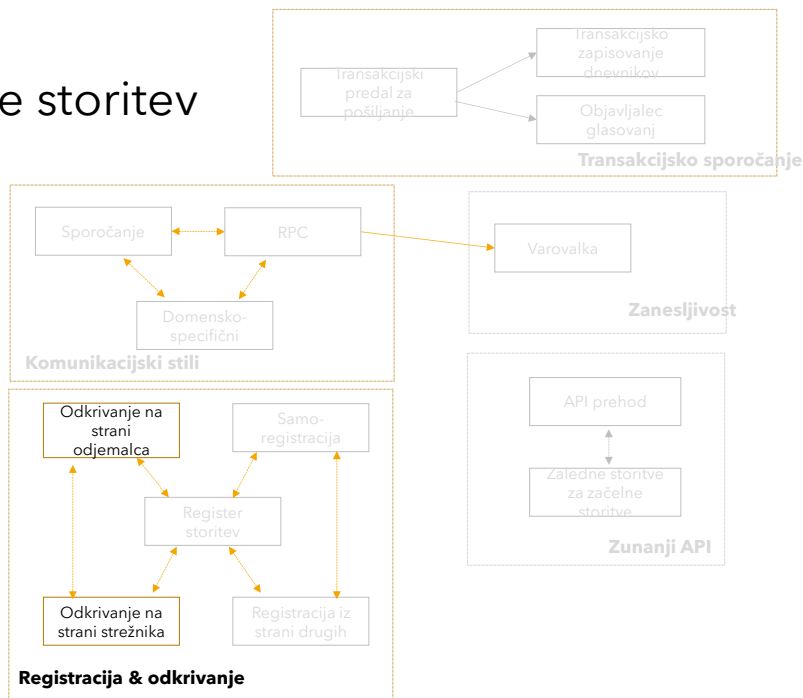
Failsafe



Resilience4j

64

Odkrivanje storitev



65

Odkrivanje storitev: osnovni izziv



- Pri klicu oddaljene storitve (npr. prek HTTP) mora odjemalec določiti **lokacijo** ciljne storitve (naslov v omrežju in vrata oziroma URL).
- Odjemalec mora torej vedeti, **kje je dosegljiv** konkreten primerek storitve.
- V mikrostoritvenih arhitekturah je to težje, ker se lokacije storitev **dinamično spreminjajo**:
 - storitve so lahko nameščene v različnih izvajalnih okoljih (oblak, gruče, zabojniki),
 - število instanc se lahko spreminja (samodejno skaliranje),
 - nove instance se lahko nameščajo na različne gostitelje.
- Zato potrebujemo mehanizme za **dinamično odkrivanje storitev** (service discovery), ki jih opisujejo različni arhitekturni vzorci.

66

Odkrivanje storitev: izzivi

samostojni odjemalec
ali
API prehod

Odjemalec
storitev

10.9.8.7:6543

VM 1

Storitev
Primerk
1

10.9.8.8:1234

VM 2

Storitev
Primerk
2

10.9.4.4:5555

VM 3

Storitev
Primerk
3

- **Naslovi (host + port) se določijo dinamično ob namestitvi storitev** – ne poznamo vnaprej.
- **Število primerkov** storitev se zaradi skaliranja **dinamično spreminja**.

67

Vzorci za odkrivanje storitev (Service Discovery)

Odkrivanje na
strani odjemalca

- **client-side discovery**

- odjemalec najprej pridobi seznam instanc storitve iz **registra storitev** in nato sam izbere, katero instanco bo poklical (pogosto vključuje tudi porazdeljevanje obremenitve).

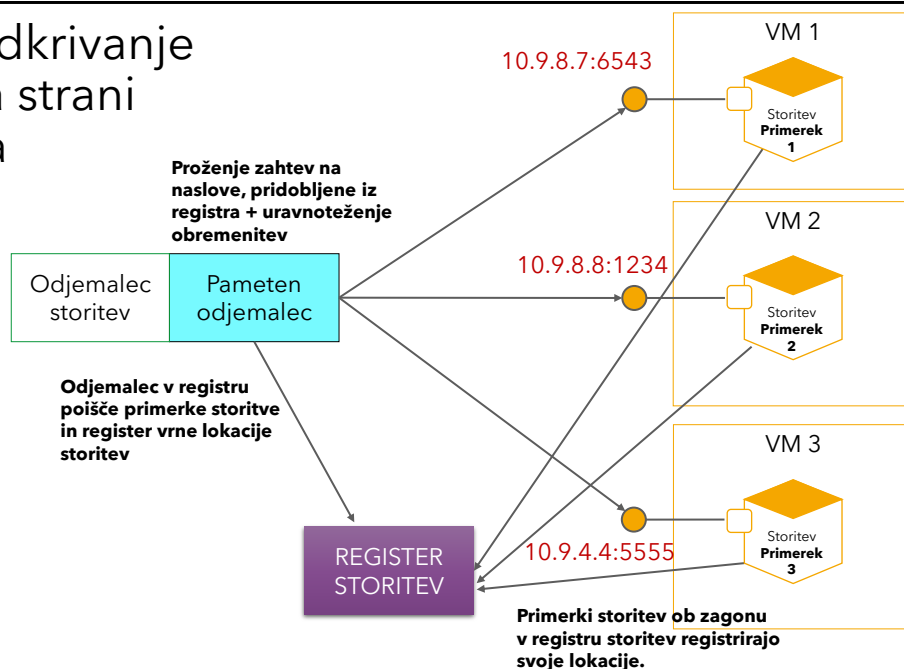
Odkrivanje na
strani strežnika

- **server-side discovery**

- odjemalec pošlje zahtevo na **posrednika** (npr. usmerjevalnik, load balancer, API prehod), ta pa na podlagi registra storitev izbere ustrezno instanco in zahtevo preusmeri naprej.

68

Vzorec: Odkrivanje storitev na strani odjemalca



69

Odkrivanje storitev na strani odjemalca: prednosti in slabosti

Prednosti

- **Večja prilagodljivost pri uravnoteženju obremenitve:** odjemalec lahko izbere instance prilagodi svojim potrebam (npr. najbližja instance, najmanj obremenjena).

Slabosti

- **Tesnejša sklopljenost** odjemalca z **registrom storitev** (odjemalec mora poznati mehanizem odkrivanja).
- Potrebna je implementacija logike za **odkrivanje storitev** in **uravnoteženje obremenitve** na strani odjemalca, pogosto v več programskih jezikih (dodatna ogrodja/knjižnice).
- **Register storitev** postane kritična komponenta, zato mora biti visoko razpoložljiv in ustrezno upravljan, sicer lahko vpliva na delovanje celotnega sistema.

70

Vzorec: odkrivanje storitev na strani odjemalca

• Primeri rešitev

- **Netflix Eureka** (<https://github.com/Netflix/eureka>)
- **Spring Cloud Netflix Eureka** (<https://spring.io/projects/spring-cloud-netflix>)
- **Netflix Ribbon** (<https://github.com/Netflix/ribbon>)
 - Spring Cloud Rest Client z Netflix Ribbon (https://cloud.spring.io/spring-cloud-netflix/multi/multi_spring-cloud-ribbon.html)
- **Netflix Zuul** (<https://github.com/Netflix/zuul>)
- ...

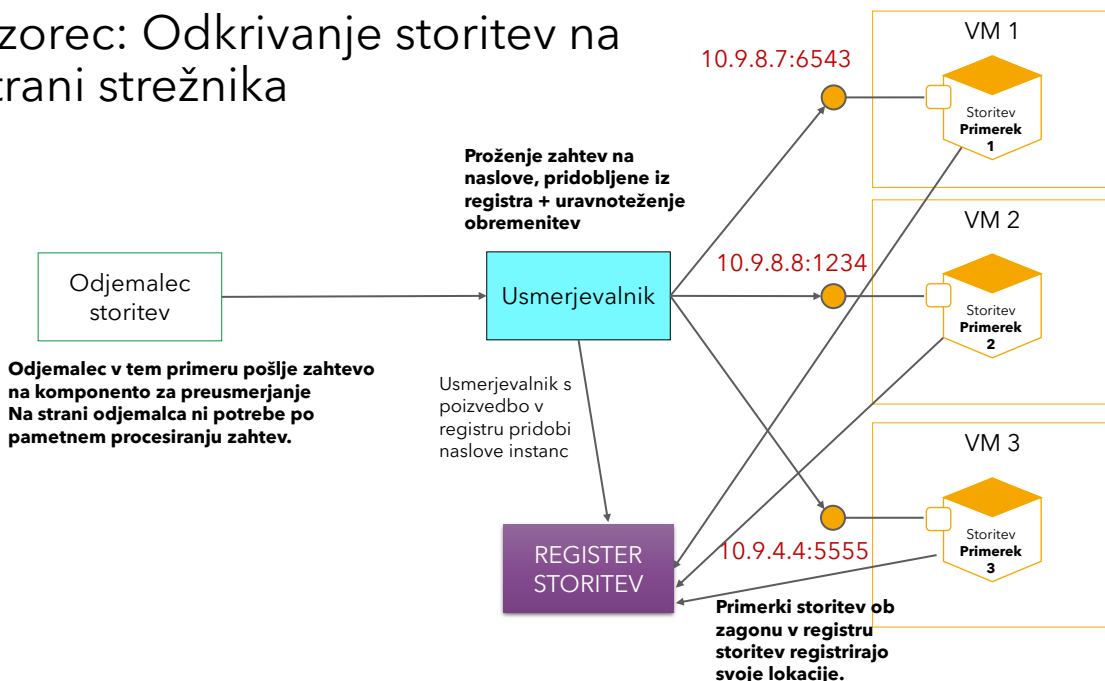


NETFLIX
RIBBON



71

Vzorec: Odkrivanje storitev na strani strežnika



72

Odkrivanje storitev na strani strežnika: prednosti in slabosti

Prednosti

- **Preprostejša koda odjemalca:** odjemalec ne potrebuje logike za odkrivanje storitev in izbiro instance.
- Pogosto je na voljo **vgrajena podpora** v platformah in okoljih, npr. **AWS Elastic Load Balancing (ELB)**, **Kubernetes**, **Marathon**.

Slabosti

- **Omejena prilagodljivost:** nabor algoritmov za uravnoteženje obremenitve je odvisen od usmerjevalnika/load balancerja.
- **Dodatne komponente:** usmerjevalnik (load balancer) in register storitev sta dodatni infrastrukturi, ki ju je treba **upravljati, nadzorovati in zagotavljati visoko razpoložljivost**.

73

Vzorec: odkrivanje storitev na strani strežnika

• Primeri rešitev

- **Amazon Elastic Load Balancing**
(<https://aws.amazon.com/elasticloadbalancing/>)



- **NGINX Plus** (<https://www.nginx.com/products/>)



74

Register storitev (Service Registry)

- **Register storitev** je komponenta, ki hrani informacije o **instancah storitev** in njihovih **lokacijah** (npr. naslov, vrata).
- Običajno podpira:
 - **registracijo** in odregistracijo storitev (ko se instance zaženejo/ustavijo),
 - **poizvedovanje** (iskanje razpoložljivih instanc za posamezno storitev).
- **Primeri rešitev**
 - **Eureka** (Netflix)
 - **Apache ZooKeeper**
 - **Consul** (HashiCorp)
 - **etcd**
 - ...
- **Opomba**
 - Okolja, kot sta **Kubernetes** in **Marathon**, pogosto že vključujejo mehanizme za **odkrivanje storitev** (npr. prek DNS/imena storitev) in s tem delujejo kot register storitev za zabojnike.

NETFLIX
EUREKA



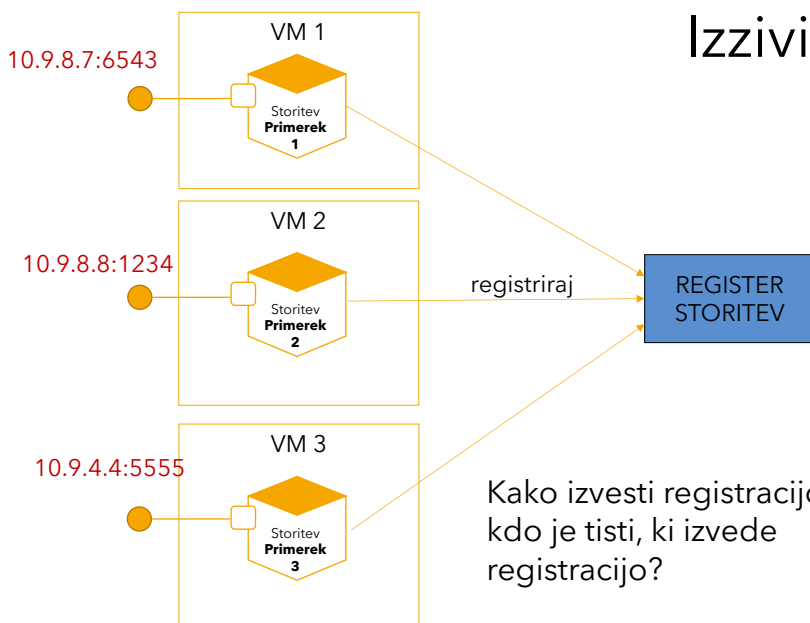
75

Zahteve glede registra storitev

- Ob zagonu mora vsaka instanca storitve izvesti **registracijo** v registru storitev.
- Ob načrtovani zaustavitvi mora instanca izvesti odjavo/izbris iz registra.
- Instance, ki odpovejo zaradi napake, je treba **samodejno odstraniti** iz registra (da jih odjemalci ne kličejo).
- Iz registra je treba odstraniti tudi instance, ki sicer tečejo, vendar niso **zdrave** oziroma ne zmorejo obdelati zahtev (npr. zaradi preobremenitve).
 - → zato je nujen mehanizem **preverjanja zdravja (health checks)** in posodabljanja stanja instanc.

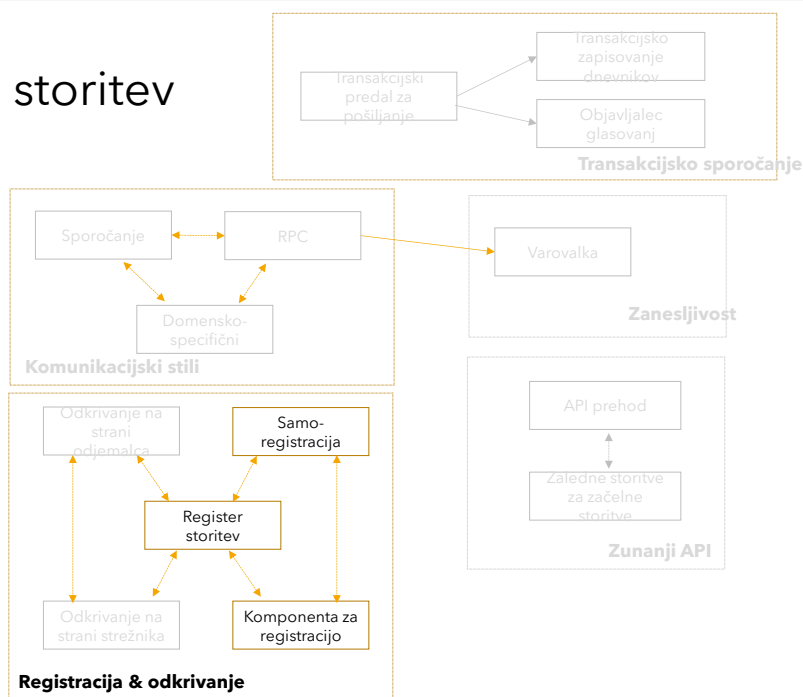
76

Izzivi, povezani z registracijo



77

Registracija storitev



78

Vzorci za registracijo storitev

Samo-registracija (self-registration)

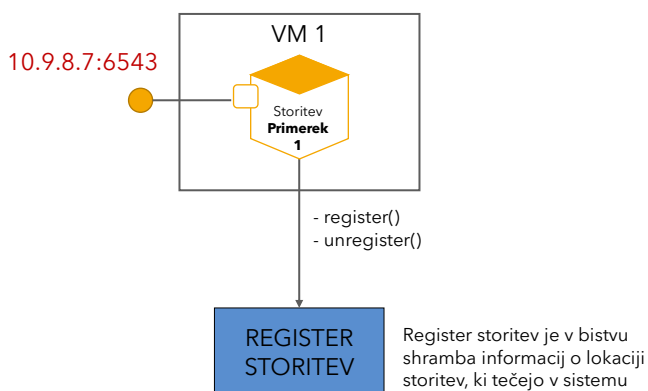
instanca storitve se ob zagonu **sama registrira** v register storitev in se ob zaustavitvi **odregistrira**.

Komponenta za registracijo (third-party registration / service registrar)

registracijo in odregistracijo izvaja **zunanja komponenta** (npr. agent, orkestrator ali platforma), storitvi pa ni treba vsebovati te logike.

79

Vzorec: samo-registracija



80

Samo-registracija: primeri rešitev

• Netflix Eureka (Java odjemalec)

- storitev se prek odjemalca ali API-ja **registrira/odregistrira** v Eureka,
- ob registraciji lahko objavi tudi **stanje instance** (npr. *STARTING*, *UP*, *DOWN*, ...),
- register izvaja **periodična preverjanja** (heartbeat/health), da zazna nedelujoče instance.



• Apache ZooKeeper (Java odjemalec)

- storitev se registrira prek odjemalca
- pogosto se uporablja pristop z **začasnimi znaki**, ki se ob prekinitvi povezave samodejno odstranijo,
- s tem je lažje zaznati instance, ki so nenadoma prenehale delovati.



81

Samo-registracija: prednosti in slabosti

Prednosti

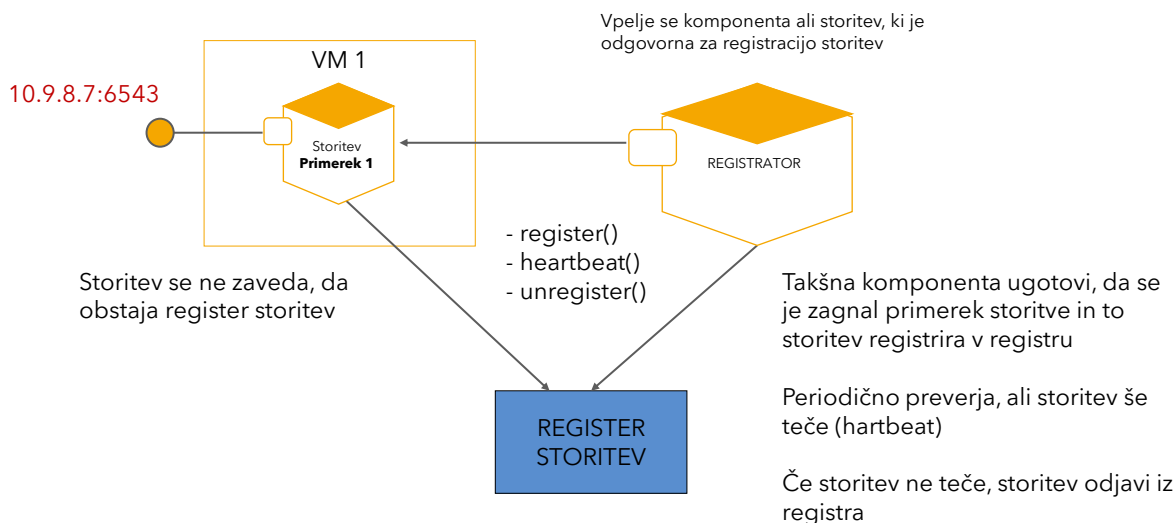
- **Preprosta zasnova:** brez dodatnih komponent; storitev se sama registrira in odregistrira.

Slabosti

- **Tesna vezanost na register storitev:** storitev mora poznati konkreten register in način registracije.
- **Podvajanje logike:** registracijo/odjavo, preverjanje stanja in ravnanje ob napakah je treba implementirati v različnih programskih jezikih (isti problem rešujemo večkrat).
- **Nezanesljiva odregistracija:** pri nenadnih izpadih se samo-odjava pogosto ne izvede, zato so nujni mehanizmi, kot so **heartbeat** in **health checks**, da register sam odstrani nedelujoče instance.

82

Vzorec: komponenta za registracijo



83

Komponenta za registracijo storitev (registrator) – primer

- **Obladne platforme (npr. AWS)** omogočajo **samodejno registracijo/odjavo** instanc (npr. EC2) v okviru platformnih mehanizmov.
- **Registrator (Gliderlabs):** agent, ki spremlja **Docker zabojnike** in jih samodejno **registrira/odregistrira** v registru storitev.
 - <https://github.com/gliderlabs/registrator>
- **Netflix Eureka Prana:** podpora za registracijo storitev v **Eureka** (pogosto v okoljih, kjer storitev sama nima Eureka odjemalca)
 - <https://github.com/netflix/Prana>
- **Buddy Works:** pomočnik/agent za **registracijo** in upravljanje povezav storitev v zabojniških okoljih.
 - <https://buddy.works/guides/docker>
- **Kubernetes / Marathon:** orkestratorja, ki vključujeta mehanizme za **odkrivanje in registracijo storitev** (npr. prek imen storitev/DNS).



Buddy Works

84

Komponenta za registracijo storitev: prednosti in slabosti

Prednosti

- **Storitve ostanejo preproste:** nimajo vgrajene logike za registracijo/odregistracijo.
- **Samodejno spremljanje stanja:** registrator lahko izvaja **preverjanje zdravja** in skrbi za posodabljanje registra.
- Nekateri oblačni platforme ponujajo registracijo »**iz škatle**« (out of the box), kar pohitri uvedbo.

Slabosti

- Registrator je **dodatna kritična komponenta**: zagotoviti je treba visoko **razpoložljivost** in zanesljivo delovanje, sicer lahko vpliva na delovanje celotnega sistema.

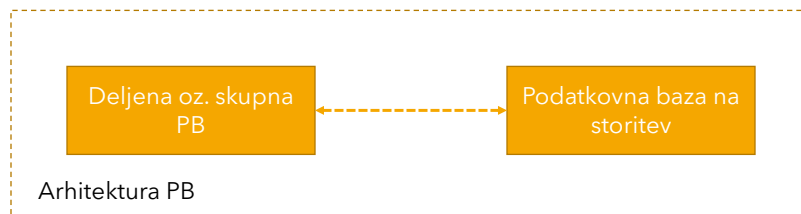
85

Mikrostoritve – podatkovne zahteve in izzivi

Poizvedbe (QUERY)	pogosto je treba združiti podatke, ki so v lasti več različnih storitev
Posodobitve (UPDATE)	poslovni procesi lahko zahtevajo usklajeno posodabljanje podatkov v več storitvah (porazdeljene transakcije)
Različne potrebe po shranjevanju	posamezne storitve lahko zahtevajo različne vrste shrambe (relacijske, dokumentne, ključ-vrednost ...)
Replikacija podatkov	zaradi skalabilnosti in razpoložljivosti je pogosto potrebna podvojitev/replikacija podatkovnih zbirk
Šibka sklopljenost	storitve morajo ostati podatkovno avtonomne , da jih lahko neodvisno razvijamo, uvajamo in skaliramo

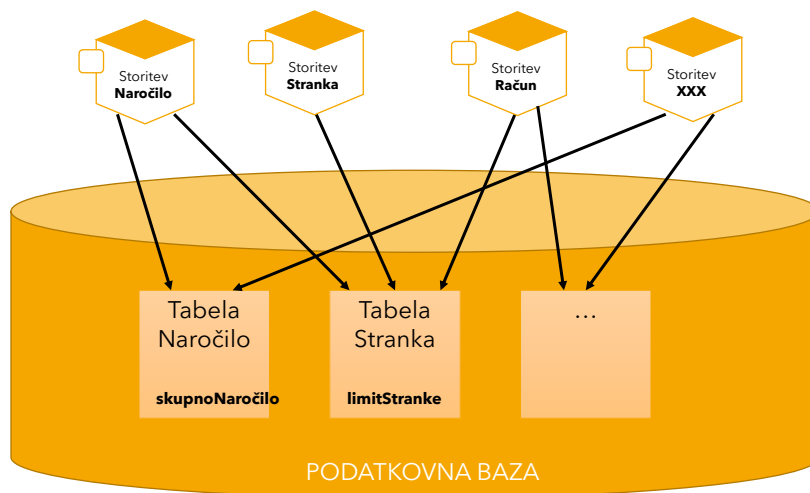
86

Upravljanje podatkov: vzorci



87

Deljena oz. skupna PB



Klasični pristop, ki se uporablja v monolitnih rešitvah

88

Deljena (skupna) podatkovna baza

Prednosti

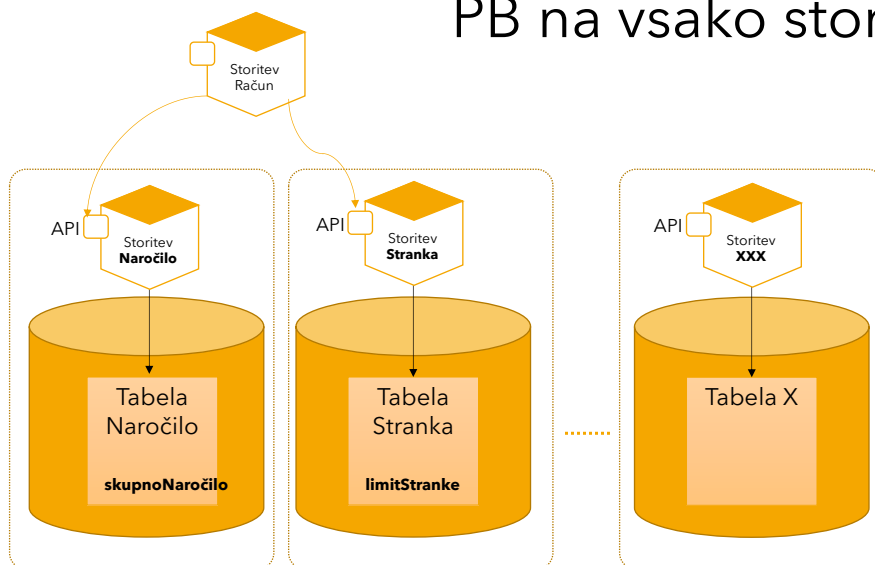
- **Enostavnejša implementacija** (enotna podatkovna plast).
- Podpora **ACID transakcijam** (*atomicity, consistency, isolation, durability*).
- **Centraliziran nadzor** nad podatki (upravljanje, varnostne politike, poročanje).

Slabosti

- **Ni enkapsulacije podatkov:** do istih tabel dostopajo različne storitve.
- **Tesna sklopljenost** storitev in posledično tudi ekip (odvisnosti med razvojnimi ekipami).
- Spremembe podatkovnega modela zahtevajo **usklajevanje** z vsemi, ki uporabljajo iste tabele → izguba avtonomije.
- Težje je slediti, **kdo uporablja katere tabele**, kar poveča tveganje pri spremembah.
- Sčasoma lahko nastane "zamrznjen" podatkovni model: spremembe so tvegane, zato se sistem **težje razvija in prilagaja**.

89

PB na vsako storitev



Vsaka storitev ima svoje private podatke, do katerih je dostop omogočen samo preko API-ja storitve!

90

Možne realizacije PB na vsako storitev

- **Privatne tabele** – storitev je lastnik množice tabel, ki so privatne za to storitev
- **Privatna shema** – storitev ima lastno shemo podatkovne baze, do katere ima dostop samo ta storitev
- **Privatni podatkovni strežnik** – storitev ima lasten podatkovni strežnik

91

Podatkovna baza na storitev (Database per Service)

Prednosti

- **Ohlapna sklopljenost:** vsaka storitev je lastnik svojih podatkov in ne deli tabel z drugimi.
- **Tehnološka prilagodljivost:** storitev lahko izbere **SUPB**, ki najbolj ustreza njenim potrebam (*polyglot persistence*), npr.:
 - relacijska podatkovna baza,
 - shramba ključ-vrednost,
 - dokumentna podatkovna baza,
 - XML shramba,
 - ...

Slabosti

- Večja **kompleksnost razvoja** (povezovanje podatkov med storitvami, porazdeljene poizvedbe).
- Težje zagotavljanje **konsistentnosti podatkov** med storitvami (pogosto *eventual consistency*).
- Klasičnih porazdeljenih transakcij tipa **2PC** praviloma ne uporabljamo/so neprimerne, zato so potrebni alternativni vzorci (npr. **saga**, kompenzacije).

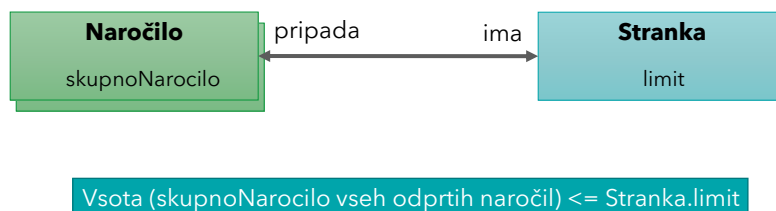
92

Konsistentnost porazdeljenih podatkov

- V porazdeljenih sistemih je zagotavljanje **konsistentnosti** velik izziv.
- Zaradi **oddaljenih in asinhronih** klicev lahko začasno nastanejo **nekonsistentna stanja** (različne storitve vidijo različne "resnice").
- Napačne poslovne odločitve na podlagi nekonsistentnih podatkov je težko analizirati, saj se posledice pogosto pokažejo **z zamikom**, ko je prvotni vzrok že skrit.
- Poslovni procesi pogosto zahtevajo posodobitev **več virov** (podatkov več storitev) v okviru ene transakcije, kar v mikrostoritvah ni trivialno.
- Zato morajo razvijalci imeti mehanizme za:
 - **zaznavanje neskladij** (monitoring, sledenje dogodkom),
 - **ponovno usklajevanje** in vzdrževanje konsistentnosti (npr. sčasoma konsistentni procesi, kompenzacije).

93

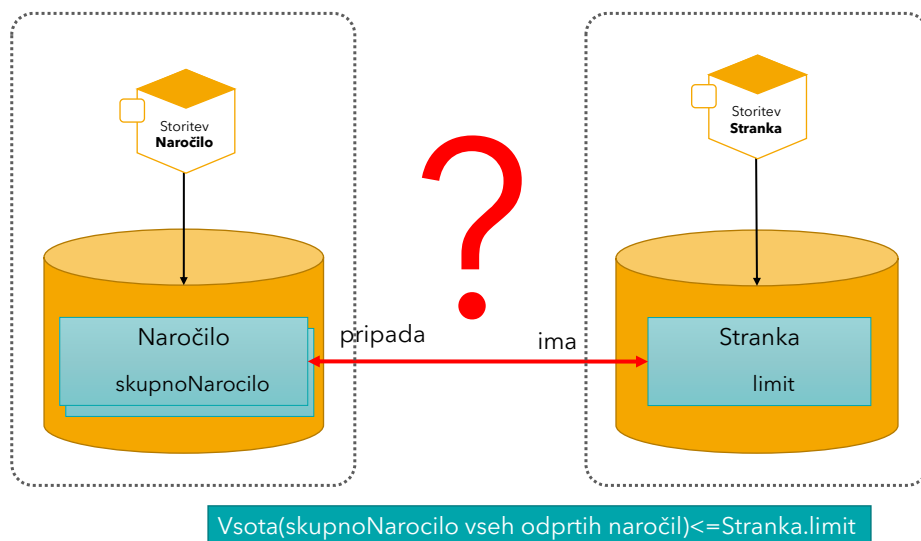
Kako vzdrževati konsistentnost podatkov brez 2PC?



- Stranka ima določen limit.
- Ko je potrebno potrditi naročilo stranke, je potrebno preveriti, da
 - novo naročilo ne preseže tega limita,
 - oz. da vsota vseh še odprtih naročil ne presega dovoljenega limita stranke.

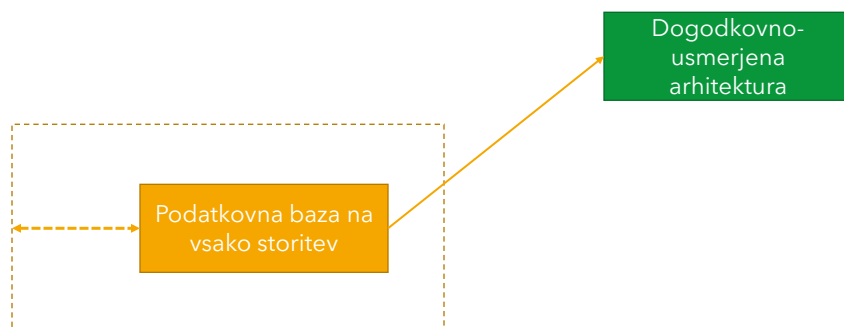
94

Kako vzdrževati konsistentnost podatkov brez 2PC?



95

Vzorec: Dogodkovno-usmerjena arhitektura



96

Kaj je dogodkovno-usmerjena arhitektura?

Event-driven Architecture (EDA)

Je pristop k izgradnji poslovne arhitekture, ki omogoča, da sistemi "slišijo" dogodke in se na njih odzovejo inteligentno.

Dogodek je nekaj pomembnega, kar se zgodi v sistemu.

97

EDA za zagotavljanje konsistentnosti

- Storitve **objavijo** dogodke, ko se zgodi nekaj pomembnega
 - npr. sprememba stanja
- Storitve so **naročene** na dogodke in posodobijo svoje stanje
 - Za vzdrževanje **sčasoma konsistentnega stanja** čez različne povezane storitve (v različnih podatkovnih bazah)
 - Za sinhronizacijo podvojenih podatkov

98

Mikrostoritve: dogodkovno usmerjeno, **sčasoma konsistentno procesiranje naročila**

